

BRANCHES OF NATURE INSPIRED COMPUTING(A) computing inspired by Nature :

(i) Nature has greatly enriched computing, Nature had been very successful in solving highly complex problems.

(ii) For low level living organisms, have to search for food, hide from predators and weather conditions, they need to mate, organise their home, etc.

(iii) All this requires complex strategies and structures which is not quite easy to understand.

(iv.) For example the procedure an ant colony foraging for food is an intelligent behaviour, this has been converted to an algorithm to solve routing problems in communication networks.

Some of the well known computing inspired by nature algorithms are:

- (i.) Artificial Neural Networks
- (ii.) Evolutionary Algorithms
- (iii.) Swarm Intelligence
- (iv.) Artificial immune systems

(B) The Simulation and Emulation of Natural Computing

This branch of natural computing provides new tools for the synthesis and study of natural phenomena that can be used to test biological theories.

There are basically two main approaches to the simulation and emulation of nature in computers:

- (i) By using tools for studying the fractal geometry of nature
- (ii) By using artificial life techniques.

Fractal geometry - There are a number of techniques for modeling fractal patterns and structures, such as cellular automata (a discrete model studied in computer science, mathematics, physics, complexity science, theoretical biology and microstructure modeling).

L-systems - Lindenmayer system is a parallel rewriting system and a type of formal grammar.

consists of an alphabet of symbols that can be used to make strings, a collection of production rules that extend each symbol into some larger string of symbols, an initial "axiom" string from which to begin construction, and a mechanism for translating the generated strings into geometric structures.

EVOLUTIONARY STRATEGIES

- i) It was developed by Rechenberg, Schwefel, and Bienenst.
- ii) used to solve parameter optimization problems.
- iii) The data structure of an ES is real-valued vectors of candidate solutions in the search plane
- iv) Let us consider the most general case of an ES.
- v) An individual $v = (x, \sigma, \theta)$ may be composed of the attribute vector x , and the sets of the strategy parameters σ and θ .
- vi) The adaption procedure is implemented performing first (the recombination and) the mutation
 - according to some probability density function of the parameters vector σ and θ ,
 - resulting in σ' and θ' , and then
 - using the updated vectors of the strategy parameters to (recombine and) mutate the attribute vector x , resulting in x' .
- vii) The pseudocode to implement an ES is the same as the EA.
- viii) It allows the evolutionary search process to adapt the strategy parameters together with the attribute vector, known as self-adaption.

ix.) Selection:

Two types of selection mechanism:

$(\mu+1)$ selection and (μ, λ) selection.

x.) In the $(\mu+1)$ ES, μ parents generate λ offspring and the whole population $(\mu+1)$ is then reduced to μ individuals.

xi.) Thus, selection operates in the set formed by parents and offspring.

xii.) In this case, the parents survive until the offspring become better adapted to the environment (with higher fitness) than their parents.

xiii.) In the (μ, λ) ES, μ parents generate λ offspring ($\lambda \geq \mu$) from which μ will be selected.

iv.) Thus, selection only operates on the offspring set.

v.) Both $(\mu+1)$ ES and (μ, λ) ES have the same general structure.

vi.) Consider the most general case of a (μ, λ) ES.

vii.) An individual $v = (x, \sigma, \theta)$ is composed of three elements: $x \in \mathbb{R}^I$, $\sigma \in \mathbb{R}^{I\sigma}$ and $\theta \in [0, 2\pi]^{I\theta}$, where I is the dimension of x ,

viii.) $I\sigma \in \{1, \dots, \ell\}$, and $I\theta \in \{0, (2I - I\sigma)$

$(\ell\sigma - 1) / 2\}$. The vector x is attribute vector.

STANDARD ANT-COLONY OPTIMIZATION (S-ACO) PROCEDURE AND VEHICLE ROUTING PROBLEM APPLICATION

- (i) Associated with each edge (i, j) of the graph there is a variable τ_{ij} termed artificial pheromone trail, or pheromone.
- (ii) Every artificial ant is capable of "making" an edge with pheromone and "smelling" (reading) the pheromone on the trail.
- (iii) Each ant traverses one node per iteration step t and, at each node, the local information about its pheromone level, τ_{ij} is used by the ant such that it can probabilistically decide the next node to move to, according to the following rule.

$$p_{ij}^k(t) = \begin{cases} \frac{\tau_{ij}(t)}{\sum_{j \in N_i} \tau_{ij}(t)} & , \text{ if } j \in N_i \\ 0 & , \text{ otherwise} \end{cases}$$

- (iv) where $p(t)_{kij}$ is the probability that ant k located at node i moves to node j ,
- (v) $\tau_{ij}(t)$ is the pheromone level of edge (i, j) , all taken at iteration t , and N_i is the set of one step neighbours of node i .

(v) In order to avoid a quick convergence of all ants towards sub-optimal routes, an evaporation of the pheromone trails was allowed.

$$T_{ij}(t) \leftarrow (1-p) T_{ij}(t) + \tau$$

where $p \in (0, 1]$ is the pheromone decay rate

vi) procedure [best] = AS-TSP (max-it, $\alpha, \beta, p, N, e, Q, T_0, b$)

initialise T_{ij}

place each ant k on a randomly selected city

let best be the best tour found from the beginning and L_{best} its length.

$t \leftarrow 1$

while $t < \text{max-it}$ do,

for $i=1$ to N do,

// build tour $T^k(t)$ by applying $(e-1)$ times the full step:

At city i , chose the next city j with probability given by Equation

end for

if a shorter tour is found,
then update best and L_{best}

end if

for every city e do,

// Update

end for

$t \leftarrow t+1$

end while

end procedure

Vehicle Routing

The main features of ACO Algorithms are the use of:

1. a colony of cooperating individuals,
2. a pheromone trail for local indirect (stigmergic) communication,
3. a sequence of local moves to find the shortest paths, and
4. a probabilistic decision rule using local information

BINARY AND REAL-VALUED NEGATIVE SELECTION ALGORITHMS

(i) There are two types of NSA

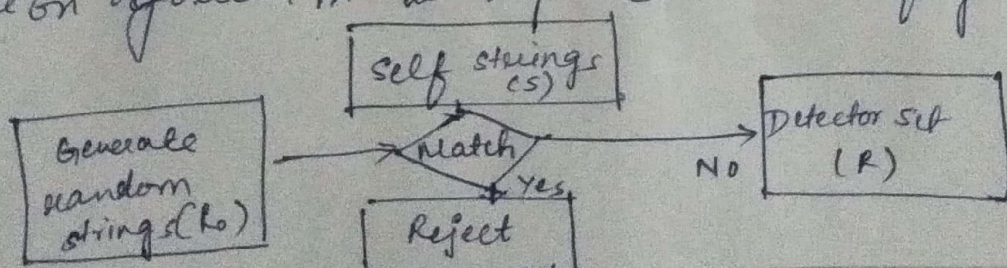
- Binary Negative selection Algorithm
- Real-valued negative selection algorithm

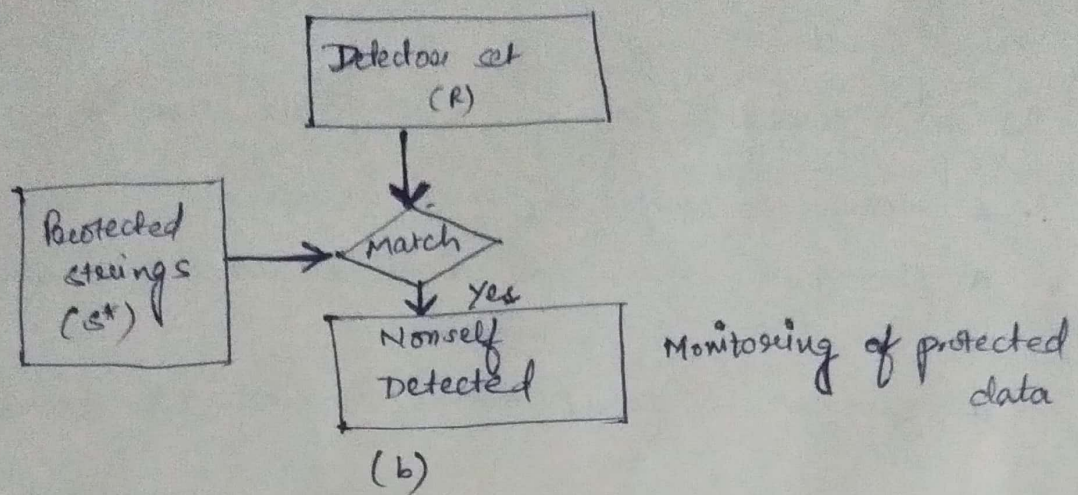
(ii) Binary Negative selection Algorithm

Forrest et al proposed a change detection algorithm inspired selection of T-cells within the thymus.

This procedure was named negative selection algorithm, and its original application was in computational security

(iii) In the censoring phase, given a known set of self patterns, the censoring phase of the negative selection algorithm is depicted in the figure





Real Valued Negative selection Algorithm :

i) RNS is rooted in same idea as original binary negative selection algorithm with the main difference that the self and nonself spaces are subsets of R^L , particularly $[0, 1]^L$.

ii) $U_{x/s} = \exp(-\|x-s\|^2 / 2\epsilon^2)$

iii) similar to the BNS algorithm, the input is a set S of self patterns, but represented by real-valued vectors instead of binary strings.

A number of detectors is initially generated at random and updated so as to cover the nonself space by moving detectors away from self vectors and keeping detectors separated from each other in order to maximise the coverage of the nonself.

HAMILTONIAN PATH PROBLEM SOLVED BY "ADLEMAN'S EXPERIMENT"

(i) Hamiltonian Path Problem :

A directed graph G with designated nodes v_{in} and v_{out} is said to have a Hamiltonian path if and only if there exists a sequence of compatible one-way edges $e_1, e_2, \dots, e_n$ that begins at v_{in} , ends at v_{out} and enters every other node exactly once.

(ii) These problems when the instance size is large, are intractable with conventional computers, but can be solved using massively parallel computers like DNA computers.

(iii) NP Problems are intractable with deterministic computers, but can be solved using non-deterministic (massively parallel) computers.

(iv) A DNA computer is a type of non-deterministic computer. The Hamiltonian Path problem was chosen by Adleman because it is known as "NP-complete".

(v) Adleman's Algorithm Input:

a) A directed graph G with n vertices, and designated vertices v_{in} & v_{out} .

b) Step 1 : Generate paths in G randomly in large quantities.

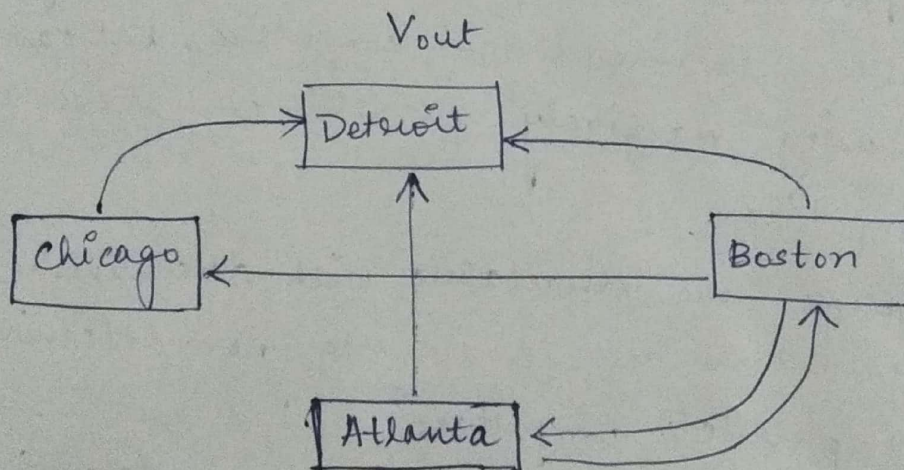
c) Step 2 : Reject all paths that
- do not begin with v_{in} and
- do not end in v_{out}

• Step-3 : For each of the vertices, reject all paths that do not involve exactly n vertices.

• Step-4 : For each of the n vertices v :
- reject all paths that do not involve n .

output : Yes, if any path remains ; No, otherwise

Example of Hamiltonian Path Problem.



Advantages:

- i) Performs millions of operations simultaneously.
- ii) Efficiently handles massive amts. of working memory.

Disadvantages:

Generating solution sets, even for some relatively simple problems, may require impractically large amts of memory.